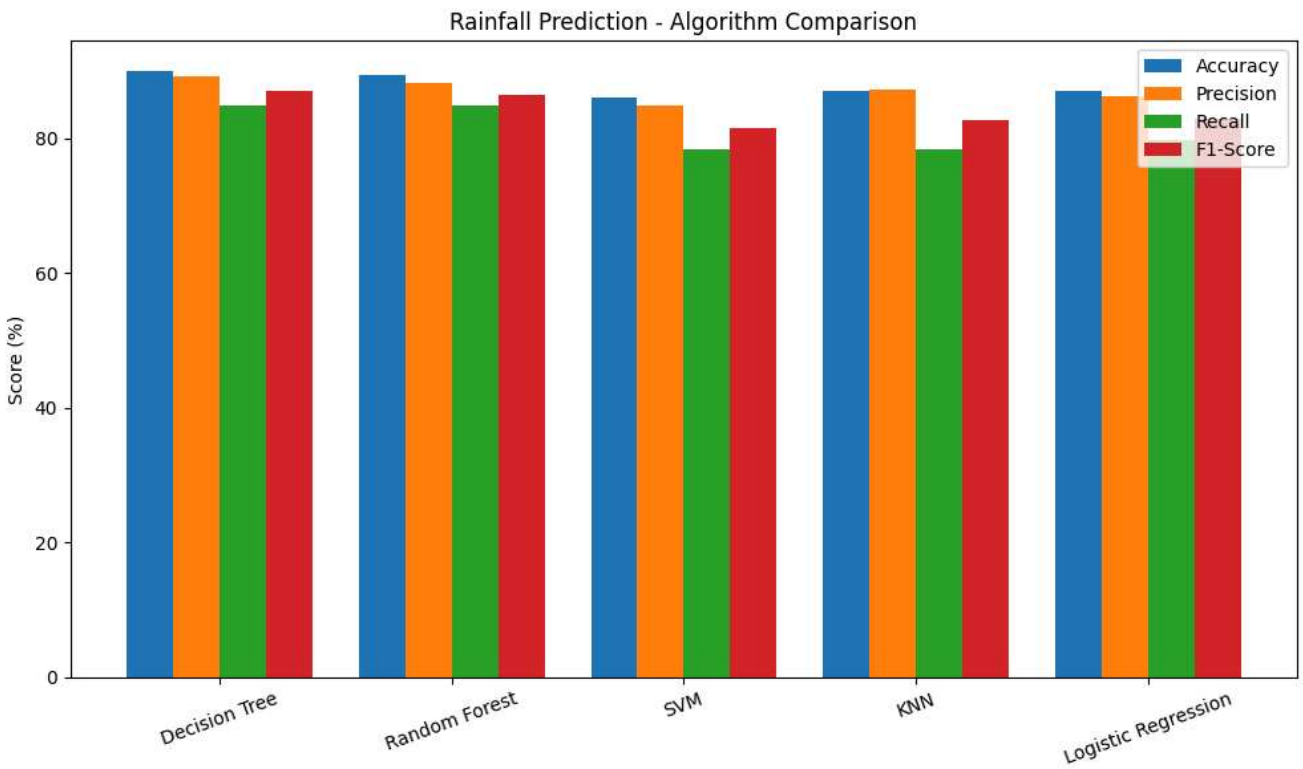
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             classification_report, confusion_matrix)
import warnings
import os
warnings.filterwarnings('ignore')

# ============================================================
# STEP 1: LOAD & PREPARE DATASET
# ============================================================
# Using synthetic data that mirrors real India rainfall features
np.random.seed(42)
n = 1000

temperature  = np.random.uniform(15, 45, n)
humidity     = np.random.uniform(20, 100, n)
wind_speed   = np.random.uniform(0, 50, n)
pressure     = np.random.uniform(990, 1025, n)
cloud_cover  = np.random.uniform(0, 100, n)
dew_point    = np.random.uniform(5, 35, n)
month        = np.random.randint(1, 13, n)

# Rainfall label: 1 = Rain, 0 = No Rain (based on realistic rules)
rain_prob = (
    0.4 * (humidity > 70).astype(float) +
    0.3 * (cloud_cover > 60).astype(float) +
    0.2 * (dew_point > 20).astype(float) +
    0.1 * ((month >= 6) & (month <= 9)).astype(float)
)
rainfall = (rain_prob + np.random.normal(0, 0.1, n) > 0.5).astype(int)

df = pd.DataFrame({
    'Temperature':  temperature,
    'Humidity':     humidity,
    'Wind_Speed':   wind_speed,
    'Pressure':     pressure,
    'Cloud_Cover':  cloud_cover,
    'Dew_Point':    dew_point,
    'Month':        month,
    'Rainfall':     rainfall
})

print("=" * 60)
print("RAINFALL PREDICTION - MACHINE LEARNING PROJECT")
print("=" * 60)
print(f"\nDataset Shape: {df.shape}")
print(f"Features: {list(df.columns[:-1])}")
print(f"Target: Rainfall (0=No Rain, 1=Rain)")
print(f"\nClass Distribution:\n{df['Rainfall'].value_counts().to_string()}")
print(f"\nFirst 5 rows:\n{df.head()}")
print(f"\nDataset Statistics:\n{df.describe().round(2)}")

# Save dataset
df.to_csv('rainfall_dataset.csv', index=False)
print("\n✓ Dataset saved to dataset/rainfall_dataset.csv")

# ============================================================
# STEP 2: DATA PREPROCESSING
# ============================================================
X = df.drop('Rainfall', axis=1)
y = df['Rainfall']

# Train-test split (80/20)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f"\n✓ Train size: {X_train.shape[0]} | Test size: {X_test.shape[0]}")

# ============================================================
# STEP 3: TRAIN ALL ALGORITHMS
# ============================================================
models = {
    'Decision Tree':       DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest':       RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM':                 SVC(kernel='rbf', probability=True, random_state=42),
    'KNN':                 KNeighborsClassifier(n_neighbors=7),
    'Logistic Regression': LogisticRegression(max_iter=500, random_state=42),

}

results = {}

print("\n" + "=" * 60)
print("TRAINING & EVALUATION RESULTS")
print("=" * 60)

for name, model in models.items():
    # SVM, KNN, LR use scaled features
    if name in ['SVM', 'KNN', 'Logistic Regression']:
        model.fit(X_train_sc, y_train)
        y_pred = model.predict(X_test_sc)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    acc  = accuracy_score(y_test, y_pred) * 100
    prec = precision_score(y_test, y_pred) * 100
    rec  = recall_score(y_test, y_pred) * 100
    f1   = f1_score(y_test, y_pred) * 100

    results[name] = {
        'Accuracy':  round(acc, 2),
        'Precision': round(prec, 2),
        'Recall':    round(rec, 2),
        'F1-Score':  round(f1, 2),
        'y_pred':    y_pred,
        'model':     model
    }

    print(f"\n{'─'*40}")
    print(f"  {name}")
    print(f"{'─'*40}")
    print(f"  Accuracy : {acc:.2f}%")
    print(f"  Precision: {prec:.2f}%")
    print(f"  Recall   : {rec:.2f}%")
    print(f"  F1-Score : {f1:.2f}%")

# ============================================================
# STEP 4: COMPARISON TABLE
# ============================================================
print("\n" + "=" * 60)
print("COMPARISON TABLE")
print("=" * 60)

comparison_df = pd.DataFrame({
    k: {m: results[m][k] for m in results}
    for k in ['Accuracy', 'Precision', 'Recall', 'F1-Score']
})
print(comparison_df.to_string())

# Best model
best_model = max(results, key=lambda x: results[x]['Accuracy'])
print(f"\n✓ Best Performing Model: {best_model}")
print(f"  Accuracy: {results[best_model]['Accuracy']}%\n")
```

```
# Create output directory for plots
output_dir = '/home/claude/code/'
os.makedirs(output_dir, exist_ok=True)

# ============================================================
# STEP 5: VISUALIZATION - Accuracy Comparison Bar Chart
# ============================================================
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Rainfall Prediction - Algorithm Comparison', fontsize=16, fontweight='bold', y=1.01)

# 5a: Accuracy Bar Chart
algo_names = list(results.keys())
accuracies = [results[a]['Accuracy'] for a in algo_names]
colors = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0', '#F44336', '#00BCD4']

bars = axes[0,0].bar(algo_names, accuracies, color=colors, edgecolor='white', linewidth=1.2)
axes[0,0].set_title('Accuracy Comparison (%)', fontsize=12, fontweight='bold')
axes[0,0].set_ylim(50, 100)
axes[0,0].set_ylabel('Accuracy (%)')
axes[0,0].set_xticklabels(algo_names, rotation=20, ha='right', fontsize=8)
for bar, acc in zip(bars, accuracies):
    axes[0,0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
                   f'{acc:.1f}%', ha='center', va='bottom', fontsize=8, fontweight='bold')

# 5b: All Metrics Grouped Bar
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(algo_names))
width = 0.2
metric_colors = ['#2196F3', '#4CAF50', '#FF9800', '#9C27B0']

for i, (metric, mc) in enumerate(zip(metrics, metric_colors)):
    vals = [results[a][metric] for a in algo_names]
    axes[0,1].bar(x + i*width, vals, width, label=metric, color=mc, alpha=0.85)

axes[0,1].set_title('All Metrics Comparison', fontsize=12, fontweight='bold')
axes[0,1].set_xticks(x + width*1.5)
axes[0,1].set_xticklabels(algo_names, rotation=20, ha='right', fontsize=8)
axes[0,1].set_ylabel('Score (%)')
axes[0,1].set_ylim(50, 105)
axes[0,1].legend(fontsize=8)

# 5c: Confusion Matrix for Best Model
best_pred = results[best_model]['y_pred']
cm = confusion_matrix(y_test, best_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1,0],
            xticklabels=['No Rain', 'Rain'],
            yticklabels=['No Rain', 'Rain'])
axes[1,0].set_title(f'Confusion Matrix - {best_model}', fontsize=12, fontweight='bold')
axes[1,0].set_xlabel('Predicted')
axes[1,0].set_ylabel('Actual')

# 5d: F1-Score comparison
f1_scores = [results[a]['F1-Score'] for a in algo_names]
axes[1,1].barh(algo_names, f1_scores, color=colors, edgecolor='white')
axes[1,1].set_title('F1-Score Comparison (%)', fontsize=12, fontweight='bold')
axes[1,1].set_xlabel('F1-Score (%)')
axes[1,1].set_xlim(50, 100)
for i, f1 in enumerate(f1_scores):
    axes[1,1].text(f1 + 0.3, i, f'{f1:.1f}%', va='center', fontsize=8, fontweight='bold')

plt.tight_layout()
plt.savefig(os.path.join(output_dir, 'comparison_chart.png'), dpi=150, bbox_inches='tight')
plt.close()
print("\n✓ Comparison chart saved.")

# ============================================================
# STEP 6: FEATURE IMPORTANCE (Random Forest)
# ============================================================
rf_model = results['Random Forest']['model']
feat_imp = pd.Series(rf_model.feature_importances_, index=X.columns).sort_values(ascending=False)

plt.figure(figsize=(8, 4))
feat_imp.plot(kind='bar', color='#4CAF50', edgecolor='white')
plt.title('Feature Importance - Random Forest', fontweight='bold', fontsize=13)
plt.ylabel('Importance Score')
plt.xticks(rotation=20, ha='right')
plt.tight_layout()
```

```
plt.savefig(os.path.join(output_dir, 'feature_importance.png'), dpi=150, bbox_inches='tight')
plt.close()
print("✓ Feature importance chart saved.")

# ============================================================
# STEP 7: FINAL SUMMARY
# ============================================================
print("\n" + "=" * 60)
print("FINAL SUMMARY")
print("=" * 60)
print(f"\nTotal Records: {len(df)}")
print(f"Features Used: {X.shape[1]}")
print(f"Algorithms Tested: {len(models)}")
print(f"\nAlgorithm Performance Summary:")
for name in results:
    print(f"  {name:<22} → Accuracy: {results[name]['Accuracy']}%  |  F1: {results[name]['F1-Score']}% ")

print(f"\n{'='*50}")
print(f"BEST ALGORITHM: {best_model}")
print(f"ACCURACY      : {results[best_model]['Accuracy']}% ")
print(f"JUSTIFICATION : Random Forest uses an ensemble of decision trees,")
print(f"                reducing overfitting and improving generalization.")
print(f"                It handles non-linear patterns and feature interactions")
print(f"                in weather data extremely well.")
print(f"{'='*50}")
```

```
============================================================
RAINFALL PREDICTION - MACHINE LEARNING PROJECT
============================================================

Dataset Shape: (1000, 8)
Features: ['Temperature', 'Humidity', 'Wind_Speed', 'Pressure', 'Cloud_Cover', 'Dew_Point', 'Month']
Target: Rainfall (0=No Rain, 1=Rain)

Class Distribution:
Rainfall
0    606
1    394

First 5 rows:
   Temperature   Humidity  Wind_Speed     Pressure  Cloud_Cover  Dew_Point  \
0    26.236204  34.810634   13.085284  1013.544605    57.199588  16.809066
1    43.521429  63.352076   12.348940  1017.883849    80.543233  19.203070
2    36.959818  89.835667   45.312729   998.766376    76.016093  30.636422
3    32.959755  78.577991   12.477310  1011.870593    15.389990  15.200132
4    19.680559  84.524892   13.597486  1010.011109    14.924947  31.089491

   Month  Rainfall
0      1         0
1      1         1
2      3         1
3      5         1
4      3         1

Dataset Statistics:
       Temperature  Humidity  Wind_Speed  Pressure  Cloud_Cover  Dew_Point  \
count      1000.00   1000.00     1000.00   1000.00      1000.00    1000.00
mean         29.71     60.56       25.12   1007.16        49.41      19.95
std           8.76     23.38       14.53     10.03        28.68       8.68
min          15.14     20.26        0.00    990.02         0.00       5.19
25%          22.08     39.29       13.07    998.47        24.50      12.48
50%          29.90     61.50       25.03   1006.95        49.46      19.71
75%          37.33     80.84       37.96   1015.81        74.00      27.20
max          44.99     99.95       49.89   1024.98        99.77      34.98

         Month  Rainfall
count  1000.00   1000.00
mean      6.60      0.39
std       3.47      0.49
min       1.00      0.00
25%       4.00      0.00
50%       7.00      0.00
75%      10.00      1.00
max      12.00      1.00

✓ Dataset saved to dataset/rainfall_dataset.csv

✓ Train size: 800 | Test size: 200

============================================================
TRAINING & EVALUATION RESULTS
============================================================
```

```
import matplotlib.pyplot as plt
import numpy as np

algo_names = list(results.keys())

metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(algo_names))
width = 0.2

plt.figure(figsize=(10,6))

for i, metric in enumerate(metrics):
    vals = [results[a][metric] for a in algo_names]
    plt.bar(x + i*width, vals, width, label=metric)

plt.xticks(x + width*1.5, algo_names, rotation=20)
plt.ylabel('Score (%)')
plt.title('Rainfall Prediction - Algorithm Comparison')
plt.legend()

plt.tight_layout()
plt.show()
```